

模擬試験 2 解答・解説

問題 1 正解：C

インタフェースの定数についての問題です。

インタフェースに宣言する変数は暗黙的に `public static final` となりますが、これらの修飾子は明示的に記述することも可能です。選択肢 A、B はアクセス修飾子に `public` 以外が指定されているためコンパイルエラーです。また選択肢 D は抽象メソッドや抽象クラスに指定する `abstract` を定数に指定しているため間違いです。なお定数宣言時の明示的な初期化は必須であり、選択肢 D、E は初期値の代入がないためコンパイルエラーとなります。選択肢 F は定数の宣言としては適切ですが定数名が小文字の `speed` となっており、これを使用する 7 行目の `SPEED` と一致せず、コンパイルに失敗します。よって正解は選択肢 C となります。

問題 2 正解：D

コマンドライン引数と暗黙の型変換についての問題です。

`args[0]` の "10" と `args[1]` の "ab" のみを取得し、以降のコマンドライン引数の値は使用していません。変数 `a` には `Integer.parseInt()` で `int` 型に変換された 10、変数 `b` には `charAt()` により取得した `char` 型の 'a' が代入されます。`a + b` の演算において、`char` 型の 'a' は暗黙の型変換により `int` 型の 97 として計算されるため、107 が出力されます。設問に指定のコマンドライン引数を渡すとプログラムは正常に実行できるため、選択肢 E、F、G の例外はスローされません。

問題 3 正解：B、D、F

基本データ型で扱えるデータサイズと型変換についての問題です。

選択肢 A の `0b0100` は 2 進数表記であり、10 進数の「4」となるため問題なく変数 `b1` に代入できます。一方変数 `b2` には `int` 型リテラルで 200 を代入していますが、`byte` 型の値の範囲である -128 から 127 を超えているため選択肢 B の代入はコンパイルエラーです。

選択肢 C は `int` 型リテラルの 300 を `short` 型に変換して代入する適切なキャストです。選択肢 D は `byte` 型と `short` 型の演算のため `int` 型で演算が行われますが、結果の受け取りが `short` 型の変数のためコンパイルエラーです。このような場合は右辺を `(short)(b1 + s1)` のように `short` 型にキャストした上で `s2` に代入するか、受け取る変数を `int` 型で宣言します。

選択肢 E は、オートボクシング／アンボクシングが行われる適切な代入です。選択肢 F は `Double` 型に `float` 型の値を代入していますが、基本データ型とラッパークラス間では暗黙の型変換のような仕組みはないためコンパイルエラーです。右辺が 20.0 であれば `Double` 型にオートボクシングされた値が `d1` に代入されます。

問題 4 正解：G

ラベルを使用した繰り返し制御についての問題です。

無限ループの for 文と、その内側に while 文、for 文の 2 つがネストされた構造があります。while 文では変数 `i` が 5 より小さい間ループし、`i` が偶数の場合は後続の処理をスキップする `continue label` があります。label が指定されているため、このときスキップする範囲は while 文の後続処理だけではなく外側の for 文の範囲となり、外側の次のループに入ります。while の条件式が `false` になると 11 行目の for 文に入り、こちらは `j==5` のときの `break label` により、外側の for 文を完全に抜ける作りになっています。しかし、for 文のカウンタ変数 `j` は、0 で初期化された値が 2 ずつインクリメントされるため、`j` の値が 5 になることはなく、無限ループとなります。

問題 5 正解：B、F

String クラスのオブジェクトについての問題です。

選択肢 A から D について、変数 `s1` は "Hello" を参照し、`s2` は `new` キーワードで新たに生成した String オブジェクトを参照しており、ここまでに 2 つの String オブジェクトが生成されています。`s2` が参照するオブジェクトはインターン化され `s3` で参照しているため、`s3` は文字列プールに存在する `s1` と同じオブジェクトを参照します。さらに `s4` も `s1` と同じオブジェクトを参照します。String クラスにも Object クラスの `toString()` がオーバーライドされていますが、既に文字列であるため自分自身のオブジェクトを返します。`s5` の参照先は、StringBuilder の `toString()` により新たに生成された String オブジェクトです。よって String オブジェクトは 3 つ生成されます。また選択肢 E、F、G については `s1 == s3` のみが `true` となります。

問題 6 正解：F

2 次元配列についての問題です。

設問の配列は、1 次元目の要素数が 2 であり、2 次元目の要素数が 3 の配列です。設問と同じ要素数の配列を生成している選択肢 A は、「`""`」による空文字の代入がないため `array[0][1]` と `array[1][2]` が `null` になり、保持する値が異なります。選択肢 B は要素に空文字を代入している点はよいですが、配列の生成時に 1 次元目の要素数を 1 つ多く指定しています。選択肢 C は `new String[3][2]` と、要素数が一致せず、値を代入している要素の位置も設問の配列とは異なります。さらに `array[1][2]` へのアクセスで `ArrayIndexOutOfBoundsException` がスローされます。選択肢 D は左辺に要素数を指定しているため、構文間違いによりコンパイルエラーとなります。そして選択肢 E も要素数の指定が設問とは異なります。よって同じ処理の配列がないため選択肢 F が正解です。

設問と同等の配列を生成する一例は、次のようになります。

```
String[][] array = new String[2][3];
array[0][0] = "A";
array[0][1] = "";
array[0][2] = "C";
array[1][0] = "X";
array[1][1] = "Y";
array[1][2] = "";
```

問題 7 正解：E

String クラスと StringBuilder クラスのメソッドについての問題です。

選択肢 A、B、C は該当せず、メソッドの使用方法はすべて適切です。replace() は String クラスと StringBuilder クラスでそれぞれ以下のメソッドが提供されています。

- String replace(CharSequence target, CharSequence replacement) : String クラスの replace()。target を replacement に置き換えた文字列を返す
- StringBuilder replace(int start, int end, String str) : StringBuilder クラスの replace()。start から end-1 の文字を str に置き換える

なお String はイミュータブルなオブジェクトのため、replace() により置き換えられた文字列は新たなオブジェクトで管理され、StringBuilder は自身のオブジェクトの文字列を変更する点にも注意しましょう。選択肢 A の処理で使用しているメソッドは String クラスの replace() ですが、変更後の文字列は参照していません。また選択肢 B で使用している StringBuilder クラスの substring() は戻り値が String 型となり、この文字列も特に参照していません。唯一、選択肢 C で使用している StringBuilder の replace() のみ、変数 sb で参照する StringBuilder のオブジェクトの文字列が "Java" となりますが、8 行目の equals() による文字列比較は、引数に StringBuilder オブジェクトが渡されているため、結果は false となります。

問題 8 正解：A、D

数値リテラルのアンダースコアについての問題です。

「_」は、桁数の大きな数値の可読性を上げる目的で使用するため、数値の間に指定できます。数値の先頭と末尾には指定できないため、選択肢 E はコンパイルエラーです。2 進数や 16 進数表記を表す 0b、0x などの接頭辞の後や、データ型の指定に使用する F や L などの接尾辞の前にも指定できないため、選択肢 B、C もコンパイルエラーです。また浮動小数点数の「.」の前後にも指定できず、選択肢

F もコンパイルエラーです。よって正解は選択肢 A、D です。数値の桁の間に使用する限り何度でも指定でき、1 つ以上連続して指定することも可能です。

問題 9 正解：G

try-with-resources についての問題です。

try-catch を使用する場合は try ブロックのみの記述はできませんが、try-with-resources では可能であり、10～12 行目に catch や finally ブロックがなくてもコンパイルエラーにはなりません。16 行目の close() に例外がスローされる処理はありませんが、throws にチェック例外が指定されているため呼び出し元で例外処理が必要です。close() の呼び出し元は try-with-resources のリソースの自動クローズ処理であり、catch ブロックを定義しない代わりに 9 行目の method() に throws を指定しています。よってプログラムは実行でき、BC が出力されます。例外がスローされず catch ブロックに制御が移らないため、6 行目の A は出力されません。

問題 10 正解：A

main() メソッドについての問題です。

java コマンドで実行するクラスを指定すると、JVM は 3 行目で宣言された main() メソッドを開始します。仮引数には String 型の配列を 1 つ用意する必要があり、可変長引数で記述することも可能なため選択肢 A が正解です。また変数名は一般的に args を使用しますが、s のように変更しても問題ありません。記述順は関係ないため選択肢 B は間違いです。

選択肢 C も間違いで、static メソッドからインスタンスメソッドを呼び出すことはできず、さらに 2 行目のメソッドとは引数リストが一致しません。2 行目のメソッドを呼び出したい場合は new Main().main("text") のように、インスタンス化した上で引数に文字列を渡して呼び出します。

選択肢 D は間違いであり、main() メソッドもオーバーロードができます。また選択肢 E、F について、public が指定されていないクラスにも同一パッケージ内からのアクセスは可能であり、コンストラクタを宣言しない場合はデフォルトコンストラクタが生成されるため、どちらもコンパイルエラーは発生しません。

問題 11 正解：D

switch 式についての問題です。

5 つの要素を持つ String 型配列を拡張 for 文でループし、その中の switch 式で該当した case の値を「+=」で count に加算していきます。なお str[2] は空文字で初期化されていますが、7 行目の処理で、str[0].replace("of", "a") により "Cafee" となり、さらに substring(0, 4) により "Cafe" となった文字列

を参照します。このため switch 式で取得する値はループの先頭から 2 2 1 1 1 であり、count は 7 となるため、選択肢 D が正解です。

「->」を使用する場合は break を記述せずに該当の case のみを処理する点も押さえておきましょう。また str[2] が空文字から変更されるため、default は実行されません。なお default と case の記述順に決まりはなく、コンパイルエラーも発生しません。

問題 12 正解：B、E

ジェネリクスについての問題です。

まず 4、5 行目では List.of() で生成したイミュータブルなリストを引数に ArrayList を生成し、変数 code で参照しています。設問においてリストの変更操作は特にありませんが、イミュータブルなリストから新たに ArrayList のオブジェクトを生成することで、ArrayList に対して変更操作が可能となります。6 行目の show() の実引数は ArrayList<String> 型のため、(1) ではこれを受け取れる仮引数の宣言を選びます。9 行目の処理において、拡張 for 文で要素を受け取る変数宣言が String 型となっていることから、ジェネリクスの型パラメータが <String> である選択肢 B、E が正解です。

選択肢 A、D は型パラメータの指定がなく、ジェネリクスの使用方法が適切ではないため、コンパイルエラーとなります。選択肢 C、F のように型パラメータを指定しない場合、要素は Object 型で扱われますが、拡張 for 文で指定されている String 型と一致しないため間違いです。

選択肢 G について、Collection、List、ArrayList などのコレクションフレームワークに所属するインタフェースやクラスの宣言には、<E> など仮の型である型パラメータが指定されているためオブジェクト生成時に必要な型を指定できますが、Object クラスはこの仕組みを持たないため、<String> のような指定はできません。

問題 13 正解：E

ソースファイルモードでのプログラム実行についての問題です。

java コマンドでソースファイル名を指定してプログラムを実行する場合は、ソースファイル内の最初のクラスが実行されます。設問では Test クラスにあたりますが、プログラムが実行できる main() が宣言されていないため、実行に失敗します。なお通常にコンパイル、実行した上で >java Main を実行すると、「3」のみが出力されます。

問題 14 正解：B

equals() と三項演算子についての問題です。

`equals()`は `Object` クラスが持つメソッドであり、サブクラスで適切にオーバーライドして使用します。`StringBuilder` クラスではこのオーバーライドがないため、6 行目の `x.equals(y)`は `Object` クラスの `equals()`が実行されて、`x` と `y` の参照先が等しい場合に `true` となります。一方 `y.equals(x)`は `String` クラスでオーバーライドされた次の定義を持つ `equals()`です。

- `boolean equals(Object anObject)` : `anObject` が該当のオブジェクトと同じ文字列の `String` オブジェクトの場合に `true` を返す

6 行目の三項演算では、`x.equals(y)`が `false` のため `y.equals(x)`が評価されますが、引数の `x` は `String` ではなく `StringBuilder` オブジェクトのため結果は `false` であり `i` には 3 が代入されます。

三項演算子を `if` 文に置き換えた選択肢の中では、`if-else if-else` の構造となっている選択肢 B が同じ処理となります。選択肢 A は `y.equals(x)`が記述されておらず条件が不足しています。選択肢 C、D は `if` に対する `else` の処理がないものや、条件式の結果に対して設定する値が異なります。選択肢 E、F は、ネストされた `if` 文となっており構造が異なります。

問題 15 正解：B、D

パッケージの使用についての問題です。

`Super`、`Sub`、`Test` クラスはそれぞれ別のパッケージに所属し、`Sub` クラスは `Super` クラスを継承し、`Test` クラスは `Sub` クラスを利用している関係です。また `Sub` クラスは `Super` クラスのインポート宣言が適切に行われています。他のパッケージのクラスを利用する際はこのようにインポート宣言を行うか、完全修飾名でクラスにアクセスを行う必要があるため、`Test` クラスの該当の対応である選択肢 D、E、F、G を確認すると、正しい記述は選択肢 D です。「`*`」が使用できるのはクラス名の部分のみのため選択肢 E は間違いであり、選択肢 F、G のように完全修飾名を使用する場合は左辺と右辺の両方に指定が必要なため、それぞれもう一方の指定が不足しています。

選択肢 A、B、C について、メンバ変数の `name` は `protected` であれば継承関係にある `Sub` クラスからのアクセスが可能です。一方 `print()`はアクセス修飾子が省略されているため、同一パッケージ内からのアクセスのみが可能です。別のパッケージに所属する `Test` クラスからもアクセスを行うためには選択肢 B の変更が必要です。選択肢 C はスーパークラスのコンストラクタ呼び出しであり、メソッド内では記述できません。

サンプルプログラムを修正した場合、一例として次のようにコンパイル、実行ができます。

```
c:¥sample¥mock2¥ex15>javac -d . *.java

c: ¥sample¥mock2¥ex15>java -cp . test.Test
Super!Sub#print()
```

問題 16 正解：G

アクセス修飾子と配列要素へのアクセスについての問題です。

メンバ変数である `countries` 配列は `private` が指定されていますが、同じクラス内からのアクセスは可能です。またクラスや `toShortName()` のアクセス修飾子は省略されていますが、`main()` 内の 12 行目で、`Country` オブジェクトを生成して `toShortName()` を呼び出す記述は適切です。よって選択肢 A、B のコンパイルエラーは発生しません。

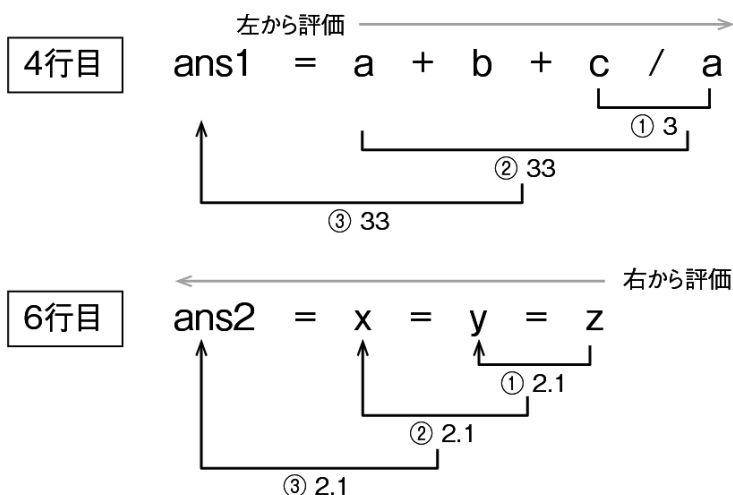
`toShortName()` は、引数で受け取った文字列の先頭から 2 文字を大文字に変換して戻します。また `main()` の処理は、`countries` 配列の要素数分ループを行い、配列要素の文字数が 2 文字の場合は次のループに入り、文字数が 2 文字でない場合は、`toShortName()` を呼び出した結果を、配列の同じ位置に再代入します。最後に配列のすべての要素を出力しているため、選択肢 G が正解です。

問題 17 正解：A

演算子と優先順位についての問題です。

`int` 型で演算を行っている 4 行目は、算術演算子と代入演算子を使用しています。代入演算子の「=」は右結合であり、演算子の右辺から評価されるため、算術演算の $a + b + c / a$ が先に評価されます。算術演算子は左から評価される左結合であり、優先度は「+」より「/」の方が高いため、 c / a の結果の「3」が $a + b$ に加算されて、右辺は「33」となります。この結果が左辺の `ans1` に代入されます。

`float` 型の演算である 6 行目はすべて代入演算子のため一番右側から評価されます。 z の値である「2.1」が y に代入され、同様に y の値が x 、 x の値が `ans2` へと代入されるため、`ans2` は「2.1」になります。



問題 18 正解：C

while 文の条件式とメソッドアクセスについての問題です。

選択肢 A は条件式が false になるためループの処理に入りません。また選択肢 B は出力の前に変数 i をインクリメントするため、「cf」が出力された後 `ArrayIndexOutOfBoundsException` がスローされます。選択肢 C は配列の要素数分ループし、i の値も 0 から 2 つずつ加算されるため、「bdg」が出力される正しい記述です。選択肢 D、E はメソッドの宣言に static がないため 4 行目から呼び出すことはできません。これらを static メソッドとした場合は、選択肢 D は無限ループの中で、i が配列の要素数よりも大きくなるとループを抜け、それ以外のときに値を出力して i の値を増やす処理は適切であり、「bdg」が出力されます。選択肢 E は最初の「b」のみ出力すると break によりループを抜けます。

問題 19 正解：D

メソッドのオーバーロードについての問題です。

11 行目以降を先に確認すると、オーバーロードされた `method()` が 3 つ定義されています。同じメソッド名でも引数の数、型、指定順が異なるものは別のメソッドであり、メソッド呼び出しの際はメソッド名と引数リストが一致するメソッドが呼ばれます。3 つの `method()` の引数リストと処理は次のとおりです。

- 11 行目：String、String の順で 2 つのオブジェクトを受け取り、どちらも "H" で始まる文字列かを検証した結果を `toShortLetter()` に渡し、その戻り値を返す
- 16 行目：String、StringBuilder の順で 2 つのオブジェクトを受け取り、どちらも "!" が含まれるかを検証した結果を `toShortLetter()` に渡し、その戻り値を返す
- 21 行目：Object、Object の 2 つのオブジェクトを受け取り、String オブジェクトに変換した上で文字数が同じかを検証した結果を `toShortLetter()` に渡し、その戻り値を返す

呼び出し元を確認すると、7 行目では String オブジェクトを 2 つ指定しているため、11 行目、16 行目の引数リストには該当せず、21 行目の `method()` が呼ばれます。String は Object クラスのサブクラスのため、暗黙の型変換により Object 型で String オブジェクトを受け取ることが可能です。続いて 8 行目では 11 行目の `method()`、9 行目では 16 行目の `method()` を呼び出します。また 25 行目の `toShortLetter()` は、3 つの `method()` から利用される private メソッドであり、true の場合は文字列の "T"、false の場合は "F" を返します。以上から、コンパイルエラーは発生せず、「TTF」が出力されます。

問題 20 正解：A

基本データ型の演算についての問題です。

6 行目で double 型の oldPrice 配列をループし、以降の else if 文で要素の値を比較した結果によって演算方法を変えて、newPrice 配列の同じ要素番号に演算結果を代入しています。

7 行目の演算の優先順は、まず代入演算子の右辺である「oldPrice[i] += 2」の演算が行われてから、最後に「newPrice[i]」への代入が行われます。8、9 行目のインクリメント／デクリメントはどちらも前置のため、同様に 1 加算または減算された後に代入が行われます。なおインクリメント／デクリメントは double 型に使用する場合も 1 ずつ増減するため、newPrice 配列の要素は選択肢 A の出力となります。

問題 21 正解：D、E、F

例外の throws についての問題です。

B の行でチェック例外がスローされるため例外処理が必須であり、try-catch で例外をキャッチするか、メソッドの宣言に throws を指定する必要があります。選択肢に try-catch に関するものはないため、throws の指定箇所を考えると、例外がスローされる methodB()、その呼び出し元の methodA()、さらに呼び出し元である main()のすべてに指定が必要となるため、選択肢 D、E、F が正解です。

問題 22 正解：A、F

クラスとインタフェースの宣言についての問題です。

選択肢 A は通常のクラス宣言であり、メソッドの宣言には「{ }」が必要ですが、「;」で終了しているためコンパイルエラーです。抽象メソッドとしたい場合は、メソッドとクラスに abstract の指定が必要です。また選択肢 F もインタフェースの default メソッドが「;」で終了しています。default メソッドは実装を持つ必要があるため「{ }」を指定するか、default のキーワードを削除します。

選択肢 B、C、D、E、G は適切です。選択肢 B は抽象メソッドを持つ抽象クラスであり、選択肢 C はシールドクラスの C と、クラス C の継承を許可された Sub クラスの関係です。選択肢 D のインタフェースは抽象メソッドが宣言され、選択肢 E のインタフェースは実装を持つ static メソッドと private メソッドの 2 つが宣言されています。最後の選択肢 G は、抽象メソッドを持つインタフェース G を継承したクラス H があり、抽象メソッドのオーバーライドはしていませんが、クラス H を abstract で宣言しているためコンパイルが成功します。

問題 23 正解：C

シールクラスについての問題です。

sealed が指定された Item インタフェースは、Food と Drink クラスに継承または実装を許可しているため、これらのクラスまたはインタフェースを適切に宣言しているものを選びます。

選択肢 A の Food インタフェースの宣言は適切ですが、Item インタフェースの permits に Drink も指定されているため、Drink クラスにも直接 Item を実装する記述が必要です。一方選択肢 C の implements には Food と Item の両方が指定されているため、選択肢 A の問題点が解消される正しい実装であり正解です。なお選択肢 C の Drink はレコードクラスのため暗黙的に final クラスです。つまりレコードの場合は sealed や non-sealed の指定はできません。

選択肢 B の Drink はインタフェースのため extends Food とすべきところ、implements が指定され、Food には permits Drink の指定も不足しているためコンパイルエラーです。選択肢 D は Food クラスを non-sealed としているため、permits の指定はできません。反対に選択肢 E は Drink クラスに sealed が指定されていますが、permits の指定がありません。また Food が final となっていますが、インタフェースには final の指定はできません。

問題 24 正解：B

2 次元配列へのアクセスと null 値の扱いについての問題です。

ネストされた for 文で 2 次元配列にアクセスし、順番に要素を出力しています。ループの条件式や var の使用はすべて適切です。出力に使用している print() は、String オブジェクトを引数に受け取る次の定義のメソッドです。

- void print(String s) : 文字列を出力する。引数が null の場合は "null" を出力する

よって引数に null を受け取っても例外がスローされることはなく、選択肢 B の出力となります。

問題 25 正解：G

オーバーライドについての問題です。

final メソッドはオーバーライドできないため、getName() でコンパイルエラーが発生します。また private メソッドはサブクラスに継承されないため、4 行目の reset() に対して 10 行目はオーバーライドではなく、サブクラスで別の reset() を再定義している関係です。3 つのメソッド宣言のうち、唯一 toString() が、適切なオーバーライドとなっています。なお、オーバーライドはインスタンスメソッド

で成立する仕組みであり、メンバ変数 `name` はアクセス修飾子に関係なく、再定義の関係となります。

17 行目の記述について、15 行目で宣言した変数 `s` は `Super` 型であり、スーパークラスで `private` 指定された `reset()` にはアクセスできないため、`(Sub(s))` とサブクラス型にキャストを行い、10 行目のメソッドにアクセスしています。

問題 26 正解：E

整数のリテラル表記と `switch` のルールについての問題です。

`Integer` 型の配列の値は 2 進数、10 進数、16 進数で表記されており、10 進数で読み替えると {3, 5, 10, 12} となります。`Calc` クラスの `operate()` は `Integer` 型を受け取り `switch` 式を実行して、結果を `int` 型で戻します。`main()` ではこの戻り値で該当の要素を書き換えた上で配列の値を出力しているため、結果は選択肢 E となります。`switch` で扱えるデータ型は、`byte`、`short`、`int`、`char` とそれらのラッパークラス、また `String` 型となる点も押さえておきましょう。

問題 27 正解：D

インスタンスと `static` 変数、参照型の扱いについての問題です。

`MyClass` のメンバ変数 `id` は `static`、`name` はインスタンスで宣言されています。5、6 行目でオブジェクトを 2 つ生成し、次に参照変数 `o2` で `id` と `name` にアクセスして値を書き換えています。このため `static` の `id` は `o1`、`o2` のどちらからアクセスしても値は "2:" となり、`name` は `o2` が参照するオブジェクトのみ "James" になります。よって 9 行目の出力は 「2:Duke 2:James」 となります。10 行目以降では、`o1` の参照先を `o2` と同じオブジェクトに切り替えた上で、参照変数 `o1` で `id` と `name` にアクセスして値を書き換えてもう一度出力を行います。よって `o1`、`o2` のどちらでアクセスしても同じ結果の 「3:Scott 3:Scott」 が出力されます。

13 行目について、`static` 変数はクラス変数とも呼ばれるように、インスタンス化せずに `MyClass.id` とアクセスを行いますが、`o1.id` のように参照を経由したアクセスも可能です。なお同じクラス内の `static` メンバである `main()` からの直接アクセスが可能なため、「`o1.id`」、「`MyClass.id`」の記述は、単に「`id`」と記述することもできます。

問題 28 正解：A

try-with-resources についての問題です。

プログラムは実行ができ、try ブロックの処理として 4 行目で SafeResource クラスの load() を呼ぶと「load:」が出力されます。続いて 5 行目の「try:」が出力され、その後 13 行目の close() が呼ばれて「close:」が出力されます。

選択肢 D、E、F、G の説明は間違っており、close() の処理で必ず例外をスローする必要はなく、チェック例外が指定されていない場合は呼び出し元での例外処理も任意です。また try-with-resources ではリソースの参照変数に var による型推論が可能であり、finally ブロックに明記しなくても、自動的にリソースのクローズ処理が行われます。

問題 29 正解：F

instanceof のパターンマッチングについての問題です。

選択肢 A、B、C について、オブジェクト型配列やパターン変数の扱いはすべて適切です。配列の要素数分ループして method() を呼び出すと、instanceof によるパターンマッチングが行われ、配列要素の順番どおりパターン変数を使用して処理が行われます。Item と Clothes オブジェクトの場合はそれぞれのメンバ変数に設定された "Item"、"Clothes" が出力されます。12 行目以降では String オブジェクトでない場合は "N/A" を出力し、String オブジェクトの場合はパターン変数 s のスコープが else ブロックとなるため、s が参照する文字列を出力します。よって、選択肢 F の出力となります。

問題 30 正解：B、E

String クラスのメソッドについての問題です。

4 行目の trim() は文字列の前後の空白を除去しますが間の空白は除去しないため、text には "Lime Lemon" の 10 文字が代入されます。なおインデックスは 0 から 9 となります。また 5、6 行目はブロックの「{」が省略された if 文であり、条件式が true となるものを選ぶと Found! が出力されます。

選択肢 A、B の indexOf() はオーバーロードされており、引数の指定はどちらも適切です。選択肢 A では最初に見つかる "m" のインデックスである 2 を取得し条件式は false となり、選択肢 B の場合はインデックス 4 以降を検索すると 7 を取得するため true となります。また選択肢 E の contains() は "m" が含まれるので true となります。

選択肢 C は文字数が異なるため false であり、選択肢 F の intern() は戻り値が boolean 型ではないためコンパイルエラーとなります。また選択肢 D は変数 text の参照とは別の文字列リテラルを用意し、参照先が等しいかを比較しているため false です。

問題 31 正解：B

ArrayList クラスのメソッドについての問題です。

A の行について、ArrayList では null 要素も管理できるためコンパイルエラーは発生しません。一方 B の行で呼んでいる add() は戻り値が void のため、変数 value1 への代入でコンパイルエラーとなります。9 行目の set() の場合は要素の変更であり、変更前の要素を取得します。また C の行の remove() はオーバーロードされていますが、いずれのメソッドも戻り値を返します (第 6 章「表 6-5」を参照)。設問のようにインデックスを指定して要素を削除すると、削除した要素を取得できます。

問題 32 正解：A、D、F

クラスの継承とインタフェースの実装についての問題です。

1 行目ではシールされたインタフェース I が宣言され、A、Base での継承または実装が必須となっています。このうち A は non-sealed クラスで宣言されて I の実装が完了しているため、未対応である Base について、適切に宣言されているものを選びます。なお 2 行目のインタフェース J は Base への実装が可能です。4 行目のクラス B は final 指定されているため、Base で継承できません。

まず必須となっているインタフェース I の実装が適切である選択肢 F は正解です。インタフェース J を実装しても構わないため選択肢 D も正解です。またクラス A の継承は可能なため選択肢 A も正解です。選択肢 B、C は構文間違いであり、extends は複数記述できません。また継承と実装を行う場合は、extends スーパークラス implements インタフェースの順となります。選択肢 E は implements に final クラスを指定しているため間違いです。選択肢 G も extends に final クラスを指定しているため間違いです。

問題 33 正解：C

do-while 文と演算子についての問題です。

main() で int 型配列の array を生成し、配列を引数に private メソッドの operate() を呼び出します。operate() では可変長引数の arr で配列を受け取ると、do-while 文により arr[i] を出力してから条件式の判定を行います。なお i は 2 行目に宣言のある static メンバ変数であり、0 で初期化されています。

条件式の論理演算子はショートサーキットを行わない論理和の「||」が使用されているため、1 つ目のオペランドの結果が true でも 2 つ目のオペランドを評価し、これにより i の値がインクリメントされています。「||」の左側の演算の中では「<=」が使用されているため、arr[i] の値が 11 のときも次のループに入ります。つまり、i の値が 2 のとき「||」の右辺の演算により i は 3 になり、次の do{} の処理で 13 が出力されます。よって 571113 までは出力される選択肢 C が正解です。なお「||」の代わり

にショートサーキットを行う「||」を使用すると i のインクリメントが行われることはなく、arr[0] の 5 が出力され続ける無限ループとなります。

問題 34 正解：D

変数のスコープについての問題です。

2 行目の static 変数 code はクラスがロードされると 3 行目の static 初期化子により初期化が行われます。4 行目のインスタンス変数 name は、main() でオブジェクトを生成すると 5 行目のコンストラクタにより "Duke" が設定されます。10 行目で呼び出している setName() は仮引数の name に "James" を代入する処理であり、メンバ変数 name の値は "Duke" から変更されていません。11 行目で println() の引数に Product オブジェクトを渡すことで、7 行目の toString() が呼ばれた結果が出力されるため、"name:" に続けてメンバ変数に設定された "Duke" が出力されます。

問題 35 正解：F

オーバーロードについての問題です。

Item オブジェクトを 3 つ生成し、Item レコードでオーバーロードされた calc() を呼び出した結果を出力しています。calc() を確認すると、12 行目と 19 行目で引数なしのメソッドが 2 つ宣言されているため、メソッド宣言が重複しコンパイルエラーが発生します。

どちらかのメソッドをコメントアウトすると実行ができ、その場合の実行結果は「200 100 110」となります。なお可変長引数は、0 個以上の引数を受け取る仕組みのため、仮に引数なしのメソッドを 2 つともコメントアウトした場合、6 行目の i1.calc() は可変長引数が指定された 13 行目にアクセスします。ただし、設問のプログラムでは、rate[0] へのアクセスで ArrayIndexOutOfBoundsException がスローされます。また 7 行目の i2.calc(0.5) の呼び出しは、可変長引数ではなく引数が完全一致する、double 型を 1 つ受け取る 16 行目の calc() が優先されます。

Math クラスのメソッドについては模擬試験 1（問題 11）でも解説しましたが、Math.floor() は小数部を切り捨てて double を戻し、Math.round() は、小数部を丸めた結果を long で戻します。

問題 36 正解：D、E

レコードクラスのコンストラクタとメソッドについての問題です。

id、author、price をコンポーネントに持ち、実行結果の出力となるよう初期化を行うコンストラクタが宣言された Book レコードを選びます。選択肢 A、F はメソッドが宣言され、どちらもコンパイルは成功しますが、選択肢 A のレコードは「author=WILLIAM,」のみの出力となり、選択肢 F のメソッドはプログラムから呼ばれていません。

残りの選択肢はすべて明示的なコンストラクタ宣言ですが、選択肢 B、C はコンパイルエラーになり、選択肢 D、E が正解となります。選択肢 B は標準コンストラクタのため、すべてのコンポーネントの初期化が必要ですが、`id` と `price` の初期化が行われていません。これを行っているものが選択肢 E です。選択肢 C はレコードのコンポーネントリストとコンストラクタの引数リストが一致しないため、適切な初期化が行われません。

選択肢 D はコンパクトコンストラクタであり、特別な処理が必要なコンポーネントの処理のみを記述することで、残りのコンポーネントの初期化も適切に行われます。コンストラクタの引数リストは記述せず、初期化処理の代入には `this.` を使用しないことが注意点です。

問題 37 正解：E

`try-with-resources` のリソースの事前定義についての問題です。

3、4 行目のように `try-with-resources` に入る前にリソースを生成する場合、その参照変数は実質的に `final` でないといけません。変数 `r1`、`r2` は明示的に `final` を指定していませんが、値の再代入はできず、`r1` は 7 行目で別のリソースを代入しているためコンパイルエラーが発生します。なお `r2` の使用方は適切です。7 行目をコメントアウトすると実行でき、その場合の出力は選択肢 B になります。リソースはオープンと逆順に閉じられること、また例外がスローされた場合もリソースを閉じてから `catch` の処理を行うことも押さえておきましょう。

問題 38 正解：A

テキストブロックについての問題です。

設問の文字列リテラルは、エスケープシーケンスにより「`"`」を文字列として扱い、変数 `text` に「`Java "17"`」を代入しています。選択肢のテキストブロックの中で同じ文字列となるのは選択肢 A です。テキストブロック内ではエスケープなしで「`"`」を記述できますが、「`""`」によるテキストブロックの終了の直前に、文字列として「`"`」を出力したい場合は「`¥`」とエスケープする必要があります。

選択肢 B は最後の「`""`」の直前に「`¥`」を指定しているため、テキストブロックの終了が認識されません。また選択肢 C はテキストブロックの開始直前に「`¥`」があるため、開始が認識されません。「`¥""`」は、テキストブロックが開始されている中で「`""`」を文字列として使用したい場合に指定します。

選択肢 D は構文としては正しいですが、終了の「`""`」の前で改行をしているため、文字列の最後に「`¥n`」が入ります。これは設問の文字列には記述されていないため、設問とは異なる文字列の指定となります。また選択肢 E、F のように開始の「`""`」と同じ行に文字列を記述すると、コンパイルエラーとなります。

問題 39 正解：D

instanceof のパターンマッチングと try-catch 文についての問題です。

Object 型を引数に受け取る validate() では、受け取ったオブジェクトのパターンマッチングが行われています。選択肢 A、B について、メソッド呼び出しは適切であり、コンパイルエラーは発生しません。B の行は実引数に int 型を渡していますが、Integer 型にオートボクシングされ、validate() の引数である Object 型に代入可能です。

11 行目の instanceof では、obj が Integer 型である場合に "Integer" が出力されます。また 14 行目の instanceof では「!」により obj が String 型ではない場合に RuntimeException をスローし、String 型の場合は、17 行目がパターン変数 s のスコープとなります。設問では変数 s は使用しておらず、"String" が出力されます。よって、4 行目の validate() の呼び出しにより「String」、5 行目の呼び出しにより「Integer」が出力された後、RuntimeException がスローされる、選択肢 D が正解です。なお RuntimeException は非チェック例外のため、validate() での throws の指定は任意です。

問題 40 正解：A、F

インタフェースのメンバアクセスについての問題です。

インタフェースの変数は暗黙的に public static final の定数となり、12 行目の (1) から、インタフェース名でアクセスします。また参照変数 p でもアクセスできるため、選択肢 C、D はコンパイルエラーにはなりません。一方インタフェースの static メソッドはインタフェース名でのアクセスのみが可能のため、選択肢 B は適切ですが、選択肢 A はコンパイルエラーになります。

4 行目で宣言している抽象メソッドは 7 行目の Product クラスで適切にオーバーライドされています。インスタンスメソッドのため参照変数名でアクセスしている選択肢 E が正しく、選択肢 F はコンパイルエラーとなります。

問題 41 正解：G

参照型と基本データ型の扱いについての問題です。

main() では Item オブジェクトの生成と int 型のローカル変数 y の初期化を行い、これらを引数に method() を呼び出しています。method() の処理では、Item オブジェクトのメンバ変数である x の値と、第 2 引数の y の値を変更した上で、11 行目でそれぞれ出力を行ってメソッドを抜けます。再び main() の処理となり、同様に 6 行目で Item クラスの x とローカル変数 y の値を出力します。

2 カ所の出力において、参照型の Item は method() と main() で同じオブジェクトを参照するため、値はどちらも同じ "xyz" となります。

基本データ型はメソッド呼び出しの際に、main()内のローカル変数 y の値が method の引数の y にコピーされ、それぞれ別の値を保持しているため異なる値が出力されます。method()からの出力は、int 型は受け取った値に 2 を加算しているため 3 となります。

問題 42 正解：E

オーバーライドについての問題です。

Super クラスと Sub クラスの execute() は、シグネチャが同一であり、戻り値の型に互換性がないためオーバーライドとオーバーロードのどちらにも該当せず、コンパイルエラーになります。

オーバーライドの場合、メソッドのシグネチャはオーバーライド元と一致させ、戻り値の型はスーパークラスと同じかそのサブクラスの型を指定します。またオーバーロードの場合、同じ名前でも異なる引数リストのメソッドを定義します。設問では Super クラス 2 行目または Sub クラス 7 行目の execute() と、Sub クラスの 10 行目の execute() がオーバーロードの関係です。

問題 43 正解：B

try-catch と Throwable クラスのメソッドについての問題です。

設問ではカスタム例外クラスのコードを割愛していますので、サンプルプログラムを確認してください。String 型の配列を引数に execute() を呼び出し、switch 文で case に該当する処理を行います。配列の 2 つ目までの要素の処理により「Open」と「Close」が出力されます。3 つ目の要素である "Friday" で DataTooLongException がスローされると、6 行目の catch ブロックに制御が移り、ex.getMessage() が呼ばれて処理が終了します。getMessage() では例外の詳細メッセージを取得しますが、特に出力はされていません。よって B が正解です。

問題 44 正解：A

ArrayList のメソッドについての問題です。

実行結果から、リストの要素数は 2 であり、2 つ目の要素の書き換えを行っていることがわかります。要素の書き換えには選択肢 A の set() を使用します。

- E set(int index, E element) : index の位置にある要素を、第 2 引数の要素で置き換える

選択肢 B の add() は指定のインデックスに要素の追加を行うメソッドです。11 行目に記述すること自体は可能ですが、実行するとリストの要素数が 3 となり、設問とは異なる出力となります。選択肢 C、D、E、F は ArrayList クラスに存在しないメソッドのため使用できません。

問題 45 正解：E

default メソッドの衝突についての問題です。

選択肢 C はクラスの継承についての説明であれば正しいですが、インタフェースは複数継承が可能のため、extends の修正は必要ありません。ただし継承元のインタフェースに同じシグネチャの default メソッドが複数存在する場合は、継承先のインタフェースでオーバーライドを行ってメソッドの衝突を回避する必要があります。したがって基本ルールとして選択肢 D は正しいのですが、設問のプログラムの場合、2 つの default メソッドは戻り値の型が異なるため、一方のメソッドのオーバーライドを行うと、もう一方のメソッドとは戻り値の型が異なる状況となり、オーバーライドは不可能です。そのため選択肢 E のどちらかのインタフェースのみを継承する修正方法が正解です。なおインタフェース A、B、C は同一ファイルで宣言されているため、選択肢 A の修正は必要ありません。また選択肢 B のような決まりはありません。

問題 46 正解：C

switch 式についての問題です。

switch の case ラベルには定数値を指定します。そのためリテラルや final 変数は指定可能ですが通常の変数は指定できないため、選択肢 C の説明は正しく、変数 i の指定はコンパイルエラーとなります。一方 final 指定された変数 j は case に指定可能なため、選択肢 D の説明は間違いです。

選択肢 A、E について、var で宣言した変数 x は int 型であるため、switch で使用できます。また例外をスローすることもできるため、default の記述は適切です。選択肢 F について、「:」を使用した switch 式では値を戻すために yield が必要であり間違った説明です。

選択肢 B について、3 & 5 はビット論理積です。「3」と「5」の 2 進数表記は以下のとおりであり、両方のビットが 1 のときに 1 となるため、10 進数表記にすると case には「1」が指定されていることになります。

- 2 進数表記（下位 4 ビット）とビット演算の結果：

3 : 0011 & 5 : 0101 → 1 : 0001

問題 47 正解：B、C

コンストラクタのオーバーロードについての問題です。

設問の Super クラス、Sub クラスは明示的にコンストラクタを宣言しているため、どちらもデフォルトコンストラクタは生成されていません。またコンストラクタの処理の先頭で別のコンストラクタを呼び出す記述がない場合はコンパイラが暗黙的に super(); を埋め込みますが、設問の Super クラス

にはこの呼び出しに対応する引数なしのコンストラクタはありません。よって必ず別のコンストラクタ呼び出しを行う記述が必要ですが、選択肢 A、F はこの記述がなくコンパイルエラーです。また選択肢 D も `super();` となっているため同じ理由でコンパイルエラーです。

選択肢 B は Sub クラスの 7 行目のコンストラクタを呼び出し、選択肢 C は Super クラスの 3 行目のコンストラクタを呼び出す正しい記述です。なお選択肢 C のコンストラクタによる初期化では、Sub クラスのメンバ変数 `code` は `null` で初期化されます。選択肢 E は、Super クラスで宣言されていない引数を 2 つ受け取るコンストラクタを呼び出しているためコンパイルエラーとなります。また選択肢 G のように、1 つのコンストラクタ内で、別のコンストラクタを複数回呼び出すことはできません。

問題 48 正解：B、F

レコードクラスのメンバについての問題です。

レコードクラスのメンバを宣言している A、B、C、D の記述のうち、B のインスタンス変数は宣言できず、コンパイルエラーとなります。A、C、D の行は `static` 変数／メソッド、インスタンスメソッドであり、これらの宣言は適切です。

Passenger レコードを利用している E、F、G の行においては、F は参照変数 `p1` で `id` にアクセスし、値を変更する処理をしていますが、コンポーネントフィールドである `id` は暗黙的に `private final` のためコンパイルエラーとなります。2 行目で宣言した `public static` な変数には `final` を指定していないため、E のようにレコード名でアクセスし値を変更する処理が可能です。G はレコードクラスに暗黙的に実装された `equals()` を使用しています。なお 10 行目の `toString()` は特に使用していません。

問題 49 正解：D

`try-catch` と `throws` についての問題です。

Shape クラスでオーバーロードされている `calcArea()` の `throws` には、1 つがチェック例外の `Exception`、もう 1 つが非チェック例外の `RuntimeException` が指定されています。どちらも例外がスローされる処理はありませんが、`throws` にチェック例外が指定されたメソッドを利用する場合は呼び出し元で例外処理が必須となります。

`main()` の処理を確認すると、14 行目で呼び出している `calcArea()` は、`throws` に非チェック例外が指定されたメソッドのため例外処理は任意であり、`catch` ブロックを用意する必要はありません。また `try-finally` のみの使用は構文上問題なく、例外がスローされない場合も必ず `finally` が実行されます。よって選択肢 D となります。

問題 50 正解：B

インタフェースの default/static/private メソッドについての問題です。

2つのインタフェース A、B を実装した MyClass では、max() をオーバーライドし、インタフェース A の max() を呼び出す記述を行って、default メソッドの衝突を回避しています。実行結果から (1) の呼び出しで 10 が返り、(2) の呼び出しで A...11 が返ることを読み取り、選択肢を確認します。10 を戻す min() はインタフェース A の static メソッドのため、インタフェース名で呼び出しを行います。選択肢 A、D のような参照名や、選択肢 C、E のような実装クラス名での呼び出しはできません。よって選択肢 B、F が候補となります。続いて (2) ではインタフェース A の max() を呼び出すため、obj.max() が指定された選択肢 B が正解です。show() については、インタフェース A 内からの呼び出しのみが可能な private メソッドであり、実装クラスから呼び出すことはできません。

問題 51 正解：C

メンバ変数のデフォルトの初期値と参照型の扱いについての問題です。

main() の処理で Product オブジェクトを 2 つ生成し p1 と p2 で参照しています。それぞれのメンバ変数は id のみコンストラクタにより初期化され、name は参照型のデフォルトの初期値である null で初期化されます。なお参照変数の型は var による型推論となっていますが、Product オブジェクトを生成することが明確なため使用方法は適切です。

11 行目では print() の引数に p1 を渡すことで、6 行目の toString() が呼ばれ、[10:null] が出力されます。13 行目の代入により、p1 と p2 は同じオブジェクトを参照するようになるため、14 行目では [20:null] が出力されます。その上で p2.setName() により name は null から "Product" に変更され、あらためて p1 を出力すると [20:product] が出力されます。

問題 52 正解：E

String と StringBuilder クラスの文字列操作についての問題です。

3、4 行目では String オブジェクトを変数 s で参照し、StringBuilder オブジェクトは sb で参照しています。ブロックの「{」が省略された if 文はネストしていない点にも注意して以降のメソッドの処理を確認すると、5 行目の if 文の条件式では s で参照する「One」と sb で参照する「Done」の文字数が一緒ではないかどうかを比較しているため true となり、6 行目の処理が実行されます。これにより、String オブジェクトの "O" を "No" に置き換えた新たなオブジェクトから「None」が出力されます。String オブジェクトはイミュータブルのため、s が参照する String オブジェクトの文字列は One から変更されていません。7 行目の if 文の条件式では s で参照する「One」と sb で参照する「Done」の文字数が一緒であるかを再び比較し、その結果を反転しているため true となり、8 行目の処理が行

われて StringBuilder オブジェクトが保持する文字列から "D" を削除した 「one」 が出力されます。よって、9 行目の出力も 「One:one」 となります。

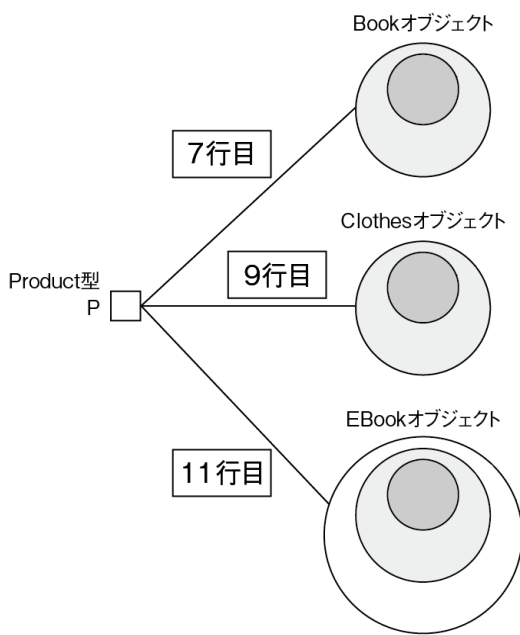
問題 53 正解：D

参照型の型変換についての問題です。

1～4 行目で宣言されたクラスは 7 行目以降でオブジェクトが生成され、Product 型の変数 p で参照されます。オブジェクトの関係は図のとおりであり、p の参照先は 7 行目で Book、9 行目で Clothes、11 行目で EBook オブジェクトへと切り替わり、13 行目で Book 型にキャストしたオブジェクトを参照します。Product 型で Book を扱うためキャストは適切であり、選択肢 A の ClassCastException はスローされません。

17 行目以降の各メソッドでは instanceof を比較演算子として使用し、p で参照するオブジェクトが instanceof の右辺の型を持つ場合に true を返します。これらのメソッドを呼び出して出力を行う処理を確認していくと次の出力が行われるため、選択肢 D が正解です。

- 8 行目：p の参照は Book オブジェクトであり EBook の実体はないため 「false」
- 10 行目：p の参照は Clothes オブジェクトのため 「true」
- 12 行目：p の参照は EBook オブジェクトであり、Clothes の実体はなく Ebook の実体はあるため 「true」
- 14、15 行目：p の参照は Book オブジェクトのため 14 行目は 「true」、15 行目は出力なし



問題 54 正解：F

パッケージとアクセス修飾子についての問題です。

com.base パッケージに Base クラス、また com.ext パッケージには Extension インタフェース、Derived クラス、Main クラスの 3 つの宣言がありアクセス修飾子は様々に指定されています。これらを利用する Main クラスでは、11 行目でコンパイルエラーが発生します。Base クラスの 5 行目で宣言されたコンストラクタを呼び出していますが、protected が指定されているため、パッケージが異なり継承関係にない Main クラスからはアクセスできません。なお Derived クラスは Base クラスと継承関係にあるため、4 行目の `super(name);` のようにしてアクセスが可能です。

仮に Base の 5 行目が `public` で宣言されている場合はコンパイル、実行ができ、選択肢 D の結果となります。Main クラス 9 行目でコンストラクタに指定した「Derived」の出力とはなりませんので注意してください。この理由は Derived クラスのコンストラクタにより Base クラスのメンバ変数 `name` に "Derived" は設定されるのですが、次の 5 行目の `print()` 内でアクセスする `name` は、`static final` である Extension インタフェースの定数であるためです。`super.name` の記述に変更し、明示的に Base クラス 3 行目のメンバ変数 `name` にアクセスしようとしても、アクセス修飾子が省略されたパッケージプライベートの変数であるためコンパイルエラーになります。

問題 55 正解：C

while 文についての問題です。

ループの条件式は `x >= -1 && y > 0` と、「&&」による論理演算が行われ、`x` と `y` の値が条件を満たす間ループの処理を行います。また「&&」の左側のオペランドでは「>=」が使用されているため、`x` の値が -1 のときもループに入ります。ループ内で `x` と `y` の値を減らす処理について、`x` は出力の前、`y` は出力の後に行われるため、`x` の値は 2、-1、-4 までは出力されます。`y` はデクリメント前のため、5、4、3 が出力されます。よって選択肢 C が正解です。

問題 56 正解：B、E、F

Throwable クラスのメソッドについての問題です。

すべての例外とエラーのスーパークラスである Throwable クラスでは、例外情報を扱うためのメソッドが提供されています。試験対策としては選択肢 B、E、F のメソッド、またコンストラクタについても API ドキュメントなどで確認しておくといでしょう。なお正解のメソッドについては第 7 章「表 7-1」でも紹介しています。その他の選択肢のメソッドは Throwable クラスには定義されていません。10 行目以降は、4 行目でスローしているカスタム例外クラスの宣言です。

問題 57 正解：D

ポリモーフィズムについての問題です。

2、3 行目で Calculator と Shape インタフェースを宣言し、4～24 行目のレコード宣言のうち、Rectangle と Triangle では Calculator のみを実装するため、calc() をオーバーライドしています。Circle には Calculator と Shape が実装されるため、calc() に加えて draw() もオーバーライドしています。これらの定義は適切ですが、プログラムは D の行でコンパイルエラーが発生する状態です。

選択肢 A について、sealed や permits の指定は可能ですが、現状でも Circle クラスのみが Shape インタフェースを実装する関係であり、特にコンパイルエラーが解消される変更ではありません。また選択肢 B は間違いであり、Circle クラスの calc() も出力に必要であり削除できません。

選択肢 C、D は instanceof のパターンマッチングです。選択肢 C は Calculator オブジェクトが Calculator の型を持つ場合にパターン変数 c を初期化していますが、常に一致するパターンマッチングのためコンパイルエラーが発生します。一方選択肢 D は正解であり、Shape を実装したクラスの場合のみ draw() を呼ぶことで、設問の出力となります。

選択肢 E について、Shape インタフェースを実装しないクラスにも、オーバーライドとは関係なく draw() を宣言することは可能ですが、Calculator には draw() の宣言がないため、35 行目の呼び出しでコンパイルエラーとなります。

問題 58 正解：C、E

戻り値の型が異なるメソッドのオーバーライドについての問題です。

選択肢から、Shape インタフェースを実装する Rectangle クラスまたはレコードのうち、calc() のオーバーライドを適切に行っているものを選びます。選択肢 A は Rectangle 単体のコンパイルは成功しますが、Shape インタフェースを実装していないため、Shape インタフェースの permits でコンパイルエラーが残ります。また implements Rectangle を付与したとしても、calc() の戻り値である Number の指定を基本データ型に変更することはできません。選択肢 B は calc() の戻り値の変更は Number のサブクラスであり適切ですが、引数リストがスーパークラスとは異なるためオーバーロードの関係となり、抽象メソッドのオーバーライドが完了していません。選択肢 D も calc() の戻り値は適切ですが、Shape を実装すべきところ、extends を使用しているため間違いです。選択肢 C、E は正解です。Long 型、Integer 型は Number のサブクラスであり、インタフェースの実装と抽象メソッドのオーバーライドも適切です。

問題 59 正解：F

インタフェースと抽象クラス、メソッドのオーバーライドについての問題です。

MyIF インタフェース、MyIF を実装した抽象クラスの MyAbstract、MyAbstract を継承した MyConcrete クラスがあります。インタフェースのメソッド z() は抽象メソッドであり、MyAbstract は抽象クラスとして宣言しているため問題ありませんが、MyConcrete クラスでオーバーライドが行われていないため、コンパイルエラーとなります。

問題 60 正解：A

例外の再スローについての問題です。

設問のプログラムには掲載されていませんが、18、19 行目で利用していることからカスタム例外クラスとして UserException と CustomException クラスがあることを認識します。10 行目で evaluate() を呼び出すと、受け取った引数「1」に応じた UserException がスローされ、evaluate() 内でキャッチします。その上で新たに Exception オブジェクトを生成し、UserException オブジェクトに設定された例外の詳細メッセージと、例外の発生原因として UserException オブジェクトを指定して Exception をスローしています。これを 12 行目でキャッチし、getCause() を呼び出した結果を出力しています。したがって、UserException に関する選択肢から、例外の原因となったクラス名が取得できている選択肢 A が正解です。